

Modular Monolith.

The pragmatic path between a tangled monolith and premature microservices.

STUDI KASUS · E-COMMERCE DEMO PROJECT

01 / 13

ARCHITECTURE · SPRING · MAVEN

Masalah pada Monolith Tradisional

Mengapa kita seringkali membenci aplikasi Monolith seiring berjalannya waktu?

- **Spaghetti Code.** Tidak ada batasan akses antar domain — logic *Order* bisa langsung mengubah tabel *User*.
- **High Coupling.** Perubahan pada satu fitur seringkali merusak fitur lain secara tidak terduga.
- **Sulit di-Scale.** Sangat sulit memisahkan satu fitur yang sibuk menjadi layanan independen.
- **Database Tightly Coupled.** Relasi `JOIN` dan `Foreign Key` saling tumpang tindih — *migration* menakutkan.

“Microservices” Bukan *Silver Bullet*

Mengapa langsung lompat ke Microservices sejak Hari ke-1 bisa menjadi bencana?

- **Kompleksitas Infrastruktur.** Membutuhkan Kubernetes, Service Discovery, API Gateway, dan teman-temannya.
- **Network Overhead.** Komunikasi antar service via HTTP/gRPC jauh lebih lambat dari sebuah *method call*.
- **Distributed Transactions.** Kegagalan di tengah transaksi membutuhkan penanganan rumit (Saga Pattern).
- **Over-Engineering.** Seringkali tim menghabiskan waktu mengurus server, bukan fitur bisnis.

SOLUSI

Modular Monolith.

Mendapatkan **kebebasan desain Microservices**, dengan **kemudahan deployment Monolith**.

- **Satu Deployment.** Tetap di-deploy sebagai satu kesatuan aplikasi — satu file `.jar`.
- **Logical Boundaries.** Kode dipisah tegas berdasarkan domain (Modul Maven).
- **Encapsulation.** Domain bisnis menyembunyikan logika dan *database*-nya dari domain lain.
- **Future-Proof.** Sangat siap dipecah menjadi Microservices saat bisnis memang sudah membutuhkannya.

Strategi Pemisahan Modul

Untuk memastikan *strict boundary*, setiap domain bisnis dipecah menjadi **2 Modul Maven**.

01 — PUBLIC

Modul Client Interface & DTO

```
ecommerce-order-client
```

Sangat ringan. Hanya berisi **interface** dan `record`/POJO.
Modul inilah yang **boleh** di-import oleh modul domain lain.

02 — HIDDEN

Modul Implementasi Logic & DB

```
ecommerce-order
```

Berisi Service, Controller, Entity JPA, dan Repository.
Tersembunyi — tidak boleh di-import oleh modul domain lain.

Pembagian Domain E-Commerce

Bagaimana aplikasi ini dipetakan menjadi modul-modul bisnis yang berdiri sendiri?

<code>ecommerce-customer</code>	Mengelola profil dan data pelanggan.	<code>ecommerce-product</code>	Katalog barang serta pemotongan / pengembalian stok.
<code>ecommerce-order</code> <code>ORCHESTRATOR</code>	Sang orkestrator — menggabungkan pelanggan, stok, tagihan, dan notifikasi.	<code>ecommerce-payment</code>	Mengelola status tagihan dan pembayaran.
<code>ecommerce-notification</code>	Pengiriman notifikasi berbasis <i>event-driven</i> .	<code>ecommerce-app</code>	Modul <i>aggregator</i> — menyuntikkan semua implementasi saat <i>runtime</i> .

Strategi Integrasi Antar-Modul

Bagaimana `Order` bisa mengurangi stok di `Product` tanpa saling mengenal?

SYNCHRONOUS

Method Call via Interface

```
ecommerce-product-client
```

Modul **Order** hanya meng-import antarmuka. Saat aplikasi berjalan, **ecommerce-app** menyuntikkan `ProductClientImpl` — hanya berupa *method call* standar JVM (sangat cepat).

ASYNCHRONOUS

Event-Driven Publisher

```
ApplicationEventPublisher
```

Modul **Notification** memisahkan *sender* dan *processor*. Klien menembakkan *event*; service mendengarkan via `@EventListener` tanpa memblokir proses utama.

Bebas dari Penjara *Foreign Key*

Relasi antar tabel modul disimpan sebagai tipe primitif — bukan sebagai @ManyToOne lintas modul.

MODUL · ORDER

orders

```
id      uuid
customer_id  varchar
product_id  varchar
amount    decimal
```



NO FK

```
CustomerClient
.getCustomer(id)
```

MODUL · CUSTOMER

customers

```
id      uuid
name    varchar
email   varchar
```

Microservice-Ready. Jika suatu hari database harus dipisah ke server fisik yang berbeda, tidak akan ada *Constraint Violation* — integritas sudah dijaga di layer logic.

Evolusi ke Microservices

Saat trafik membludak dan modul `Product` harus dipisah ke server lain — bagaimana caranya?

- 01 **Kode di Order tidak diubah.** Pemanggilan `productClient.reduceStock()` di `OrderService` sama sekali tidak tersentuh.
- 02 **Ganti implementasi client.** `ProductClientImpl` diganti menjadi `ProductClientRestImpl` — memanggil REST API via `RestTemplate` atau `FeignClient`.
- 03 **Pindahkan modul Product.** Deploy modul ke server / container barunya.
- 04 **Selesai.** Migrasi dalam hitungan hari, bukan bulan.

Perlindungan Arsitektur & Dokumentasi

Mencegah pelanggaran *tight coupling* sebelum kode masuk ke *production*.

01 — ARCHITECTURE TEST

ArchUnit

```
build pipeline gate
```

Mengawal agar batas antar-modul tidak ditrabas oleh baris kode baru. **Build otomatis gagal** jika Modul A meng-import kelas implementasi Modul B tanpa lewat *Client Interface*. Investasi arsitektur tetap berumur panjang — tidak membusuk kembali menjadi *Spaghetti Monolith*.

02 — API DOCS

Swagger OpenAPI

```
/swagger-ui.html
```

Aplikasi secara otomatis menyajikan dokumentasi gabungan dari **seluruh modul** di satu UI. Tampilan API utuh layaknya berasal dari sekumpulan Microservices sungguhan.

Praktek Langsung Keunggulan Arsitektur

■ main

Arsitektur stabil, *clean*, dan lulus semua tes.

■ demo/architecture-test PR #1

Mensimulasikan pelanggaran *tight coupling* — modul **Order** memanggil implementasi modul **Product**. Build digagalkan otomatis oleh ArchUnit.

■ demo/split-module-as-microservices PR #2

Modul **Payment** dihapus, diubah jadi pemanggilan `RestClient` via HTTP. Tidak ada perubahan apa pun di `OrderService` .

■ demo/event-driven-architecture PR #3

Modul **Notification** migrasi dari Spring Application Event ke **Apache Kafka**. Sekali lagi — tidak ada perubahan di `OrderService` .

KESIMPULAN

Batu pijakan untuk hampir semua tim.

Modular Monolith memberi struktur tanpa pajak distribusi — cocok untuk *startup* maupun fitur baru.

Fokus pada bisnis

Tidak pusing infrastruktur di awal — semua effort diarahkan ke fitur.

Jaring pengaman

Terlindungi penuh oleh *end-to-end* integration test antar modul.

Strict boundary

Kode tetap bersih dan rapi layaknya Microservices.

Jalan keluar jelas

Perpindahan ke *distributed system* sudah dipetakan dengan gamblang.

Q&A

Terima kasih.

DEMO PROJECT • MODULAR MONOLITH